

Project 1, Session 3: Data storage and reproducibility

- Paired team review and activity
- Principles for storing data
- Principles for reproducible analyses

Team discussion

Paired teams: 4-6-8, 1-5, 2-9, 3-7

- Explain what each row and column means in the score tables
- How did you parse the mutation labels in the score tables to identify the site, WT residue, and mutation?
- How did you verify that the listed WT residue matches the reference?
- How did you compare score distributions for WT, missense, nonsense, and synonymous mutations, and what do they reveal?

When presenting, make sure to **clearly describe your code** and what it does.

Team activity

Could your parser catch labels with incorrect formatting or unexpected values?

Let's find out.

Together with your team, prepare an input file in the same format as the PTEN scores, but with issues with some mutation labels that would warrant a manual review. This could just be a modified version of the original score file.

Prediction sheet A: record the concerning labels and why they should be flagged by the parser.

Exchange your input file with your partner team. What happens when you attempt to parse their file? After processing the file, describe the issues that your code found, and explain any concerning labels that your partner team's code did not identify.

Prediction sheet B: record which concerning labels, if any, that your team's code wasn't able to detect, and how it could be modified to catch them.

Checking input data for consistency is not just an academic exercise

Real datasets can contain errors or violate implicit assumptions.

Examples from real life:

- sample IDs with numbers transposed
- duplicated replicate numbers for what should be different replicates of an experiment
- a sample with a collection date in 2165

Guiding principles for storing data

Why store data?

Analysis -- we want to learn something from the data

- By ourselves, for our own work
- To share with others, along with interpretations

Record keeping

To meet these needs, data should be **easy to interpret** and **easy to use**

A quick individual activity

Prediction sheet C: What does this data set describe? Record your reasoning.

<https://mscbio2025-2026.github.io/data/test.csv>

A case study

... a realistic example of tabular data in the wild: It contains redundant columns, odd variable codes, and many missing values. In short, [this dataset] is messy.

Garrett Grolemund

(<http://garrettgman.github.io/tidying/>)

What *does* it mean?

This data set describes confirmed cases of tuberculosis (TB) gathered by the WHO
Cases are sorted by country, age, and sex

A better way: tidy data

Organizing principles for storing data (see work by Hadley Wickham)

- Each variable forms a column
- Each observation forms a row
- Each value is placed in its own cell

Tidy data

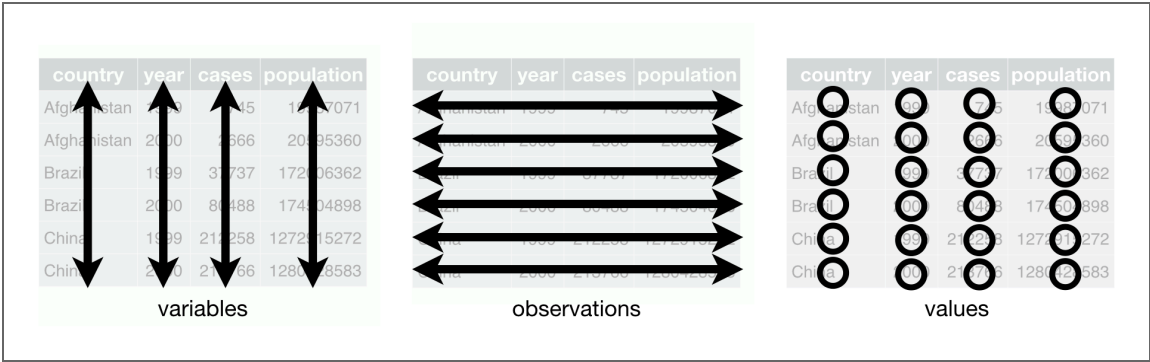


Image: Garrett Grolemund

Principles for reproducible analyses

How many people have tried to reproduce someone else's analysis from a published paper?

Was it easy?

It should be! We want our analysis to be **easy to reproduce** and **easy to share**.

Reproducing our PTEN analysis

Imagine that another team working with your code and the same published data.

What else could change the result when they rerun it?

A useful record includes:

- The exact input files and their source.
- The code, filtering rules and other parameters.
- Python and package versions.
- A fresh run and checks on intermediate results.

The Python behind the notebook

The **kernel** is the running process that executes our cells.

Its **environment** contains a Python interpreter and the packages available to it.

Inspect: Which Python is this notebook using?

```
import sys

print("Python version:", sys.version.split()[0])
print("Python location:", sys.executable)
```

A version report in the notebook

At the end of an analysis, `session-info` can report imported modules, Python and the operating system.

`dependencies=True` also includes modules imported by other packages.

```
import pandas as pd
import matplotlib.pyplot as plt
import session_info

session_info.show(dependencies=True)
```

Save the notebook with the report output retained.

A snapshot of installed packages

`conda env export` writes installed package versions in a format that conda can read later. For example, let's say that you create a virtual environment called `pten-project` for packages specifically related to this project.

```
conda activate pten-project
conda env export > environment.yml
```

Open `environment.yml` to inspect the saved list. It includes unused packages and dependencies. Projects often call their saved snapshot `environment.yml` or `requirements.txt`.

A virtual environment for each project

A **virtual environment** gives a project its own set of installed packages.

PTEN project environment	Another project environment
Packages for our abundance analysis	Packages for a different analysis
Updates stay in this environment	Updates stay in this environment

An environment with only the project's tools produces a more focused package list that won't be broken by updates to packages in another environment.

Recreating a stored environment

Installing the necessary packages (and the right versions!) from a saved `environment.yml` file is easy.

```
conda env create -n pten-reproduced -f environment.yml  
conda activate pten-reproduced
```

Best practices for sharing an analysis

- Preserve the input files and record their source.
- Save a software snapshot alongside the code.
- Record Python, the working folder, and rerun steps in a README file.
- If using notebooks: restart, run all cells, and save the notebook with its version report.

Assignment for next time

With your team, investigate abundance across PTEN positions, using all checked missense variants:

- Describe coverage (number of observed missense variants) across all 403 positions, including positions with no measured substitutions.
- Compute score statistics for each position with at least `minimum_variants = 5` included substitutions. Keep missing scores distinct from zero.
- Write test functions to verify that your code to compute abundance and score statistics performs as expected and flags or rejects unexpected information.
- Visualize coverage and score statistics across the PTEN protein sequence.

Switch to the next role!